Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

//--- Function Prototypes ---
void display(int arr[], int size);
void swap(int *a, int *b);

// Sorting Algorithms
void mergesort(int arr[], int left, int right);
void quicksort(int arr[], int low, int high);

// Helper functions
void merge(int arr[], int l, int m, int r);
int partition(int arr[], int low, int high);


//--- Main Program Logic ---
int main() {
    int n = 0, option;

    while (n <= 0) {
        printf("Enter the number of elements for the array (must be > 0): ");
        scanf("%d", &n);
        if (n <= 0) {
            printf("Invalid size. Please try again.\n");
        }
    }

    int original_arr[n];
    int arr_to_sort[n];

    printf("\nEnter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &original_arr[i]);
    }

    clock_t start, end;
    double cpu_time_used;
```

```
while (1) {
    printf("\n--- Sorting Menu ---\n");
    printf("1. Merge Sort\n");
    printf("2. Quick Sort\n");
    printf("3. Exit\n");
    printf("Choose an option: ");
    scanf("%d", &option);

    // Before every sort, create a fresh copy of the original array
    for(int i = 0; i < n; i++) {
        arr_to_sort[i] = original_arr[i];
    }

    if (option >= 1 && option <= 2) {
        printf("\nOriginal array: ");
        display(original_arr, n);
    }

    switch (option) {
        case 1:
            start = clock();
            mergesort(arr_to_sort, 0, n - 1);
            end = clock();
            cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
            printf("Array sorted with Merge Sort: ");
            display(arr_to_sort, n);
            printf("⏰ Merge Sort took %f seconds.\n", cpu_time_used);
            break;

        case 2:
            start = clock();
            quicksort(arr_to_sort, 0, n - 1);
            end = clock();
            cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
            printf("Array sorted with Quick Sort: ");
            display(arr_to_sort, n);
            printf("⏰ Quick Sort took %f seconds.\n", cpu_time_used);
            break;

        case 3:
            printf("Exiting program. Goodbye!\n");
            exit(0);
```

```
            default:
                printf("Invalid option! Please choose between 1 and 3.\n");
        }
    }

    return 0;
}

//--- Function Definitions ---

void display(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

//--- Merge Sort Implementation ---
void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    // Create temporary arrays
    int L[n1], R[n2];

    // Copy data to temp arrays L[] and R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    // Merge the temp arrays back into arr[l..r]
    i = 0; // Initial index of first subarray
    j = 0; // Initial index of second subarray
    k = l; // Initial index of merged subarray
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[], if there are any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[], if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergesort(int arr[], int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;
        // Sort first and second halves
        mergesort(arr, left, middle);
        mergesort(arr, middle + 1, right);
        // Merge the sorted halves
        merge(arr, left, middle, right);
    }
}

//--- Quick Sort Implementation ---
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // pivot
    int i = (low - 1);     // Index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than the pivot
```

```
        if (arr[j] < pivot) {
            i++; // increment index of smaller element
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quicksort(int arr[], int low, int high) {
    if (low < high) {
        // pi is partitioning index, arr[p] is now at right place
        int pi = partition(arr, low, high);
        // Separately sort elements before and after partition
        quicksort(arr, low, pi - 1);
        quicksort(arr, pi + 1, high);
    }
}
```

Sample Execution Output

```
Enter the number of elements for the array (must be > 0): 7

Enter 7 elements:
45
12
89
7
55
23
61

--- Sorting Menu ---
1. Merge Sort
2. Quick Sort
3. Exit
Choose an option: 1

Original array: 45 12 89 7 55 23 61
Array sorted with Merge Sort: 7 12 23 45 55 61 89
⏰ Merge Sort took 0.000004 seconds.
```

--- Sorting Menu ---
1. Merge Sort
2. Quick Sort
3. Exit
Choose an option: 2

Original array: 45 12 89 7 55 23 61
Array sorted with Quick Sort: 7 12 23 45 55 61 89
⏰ Quick Sort took 0.000002 seconds.

--- Sorting Menu ---
1. Merge Sort
2. Quick Sort
3. Exit
Choose an option: 3
Exiting program. Goodbye!

Algorithms and Analysis

Overall Program Algorithm

This describes the flow of the main application.

1. **Start** the program.
2. **Initialize Variables**: Declare variables for array size (n), user option (option), and timing.
3. **Input Array Size**: Prompt the user to enter the number of elements for the array.
4. **Validate Input**: Loop until the user enters a size greater than 0.
5. **Declare Arrays**: Create two integer arrays of size n: original_arr and arr_to_sort.
6. **Input Array Elements**: Prompt the user to enter the elements and store them in original_arr.
7. **Start Main Loop**: Begin an infinite while loop to display the menu.
8. **Display Menu**: Show the user the options: 1 for Merge Sort, 2 for Quick Sort, and 3 to Exit.
9. **Get User Choice**: Read the integer option chosen by the user.
10. **Create Array Copy**: Before performing any sort, copy all elements from original_arr to arr_to_sort.
11. **Process Choice (Switch Statement)**:
    - **Case 1 or 2**:
      - Record the start time using clock().
      - Call the corresponding sort function (mergesort or quicksort) with arr_to_sort.
      - Record the end time using clock().
      - Calculate the cpu_time_used in seconds.
      - Print the sorted array and the time taken.
    - **Case 3**: Print an exit message and terminate the program using exit(0).
    - **Default**: If the choice is invalid, print an error message.
12. **Loop Continuation**: The loop automatically repeats from Step 8.

Merge Sort

Algorithm:
Merge Sort is a "divide and conquer" algorithm.

1. **Divide**: If the array has more than one element, divide it into two halves: a left half and a right half.
2. **Conquer**: Recursively call mergesort on the left half. Then, recursively call mergesort on the right half. This continues until the sub-arrays have only one element (which are inherently sorted).
3. **Combine (Merge)**: Once the two halves are sorted, merge them back together. To do this, create two temporary arrays to hold the left and right halves. Compare the first

elements of both temporary arrays, copy the smaller one into the original array, and advance the index of the temporary array from which the element was taken. Repeat this process until one of the temporary arrays is empty. Finally, copy the remaining elements from the non-empty temporary array into the original array.

Complexity Analysis:

- **Time Complexity:** O(nlogn) for all cases (Worst, Average, and Best). The algorithm always divides the array in half and takes linear time to merge the halves.
- **Space Complexity:** O(n). Merge sort requires additional space proportional to the size of the input array for the temporary arrays used during the merge step. It is not an in-place algorithm.

Quick Sort

Algorithm:
Quick Sort is another "divide and conquer" algorithm that is highly efficient on average.

1. **Partition**:
    - Choose an element from the array to be the **pivot**. A common choice is the last element.
    - Rearrange the array so that all elements smaller than the pivot are placed before it, and all elements greater than the pivot are placed after it. The pivot is now in its final sorted position.
    - This rearranging is done in the partition function, which returns the final index of the pivot.
2. **Conquer**:
    - Recursively call quicksort on the sub-array of elements to the left of the pivot.
    - Recursively call quicksort on the sub-array of elements to the right of the pivot.
3. **Base Case**: The recursion stops when a sub-array has zero or one element, as it is already sorted.

Complexity Analysis:

- **Time Complexity:**
    - **Worst Case:** O(n2). This occurs when the pivot element is always the smallest or largest element, leading to an unbalanced partition (e.g., on an already sorted array).
    - **Average Case:** O(nlogn). This is the most common scenario.
    - **Best Case:** O(nlogn). Occurs when the partition process always divides the array into two equal halves.
- **Space Complexity:** O(logn) on average (for the recursion stack). In the worst case, it can be O(n).